

From IDEs to Automation for Advanced Programmers



Instructor: Nabajyoti Medhi

Advanced Programming by Nabajyoti Medhi

[illegible]

Agenda for the Session



Programming Ecosystem

- IDEs (VS Code, JetBrains)
- Build Systems (Make, Maven, Gradle)
- Debugging & Profiling
- Source Code Management (SCM)



Command-line Usage

- Navigation (cd, ls, pwd)
- File Operations (cp, mv, rm, mkdir)
- Pipes & Redirection (|, >, <)
- Help Systems (man, --help)



Git Workflows

- Local & Remote Repos (init, clone, push, pull)
- Branching & Merging (branch, checkout, merge)
- Collaboration & Pull Requests
- Resolving Conflicts



Shell & Bash Scripting

- Shell Environments (Bash, Zsh)
- Variables & Conditionals (if, else, case)
- Loops & Functions (for, while)
- Practical Scripts (automation tasks)



Version Control (Git)

Editor/IDE

Compiler/Interpreter

Build System

Debugger

Profiler

Your Code

- A **programming ecosystem** is the collection of tools used to edit, build, run, test, debug, and version code in a consistent workflow.
- Includes Editor/IDE, compiler/interpreter, build system, debugger, profiler, and version control.
- Good ecosystems reduce friction so developers focus on solving problems instead of fighting tools.

IDEs vs Plain Editors

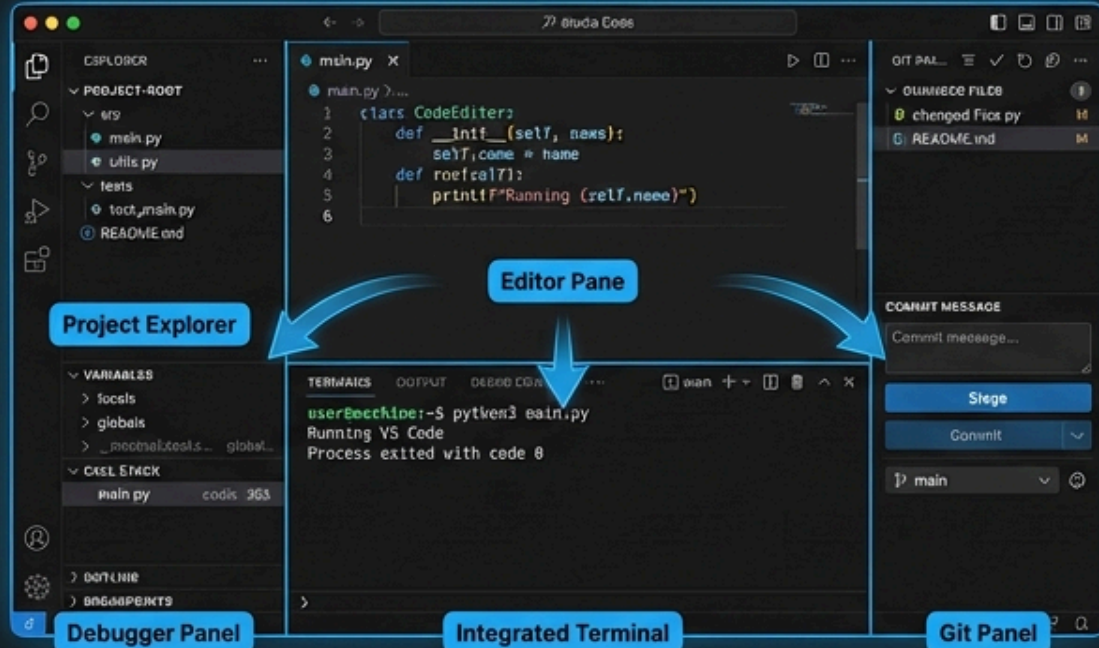
1. **IDE:** All-in-one tool with integrated features.

2. **Plain Editor:** Minimalist tool, combine with separate tools.

3. **Trade-off:** IDEs offer convenience; Editors provide flexibility.

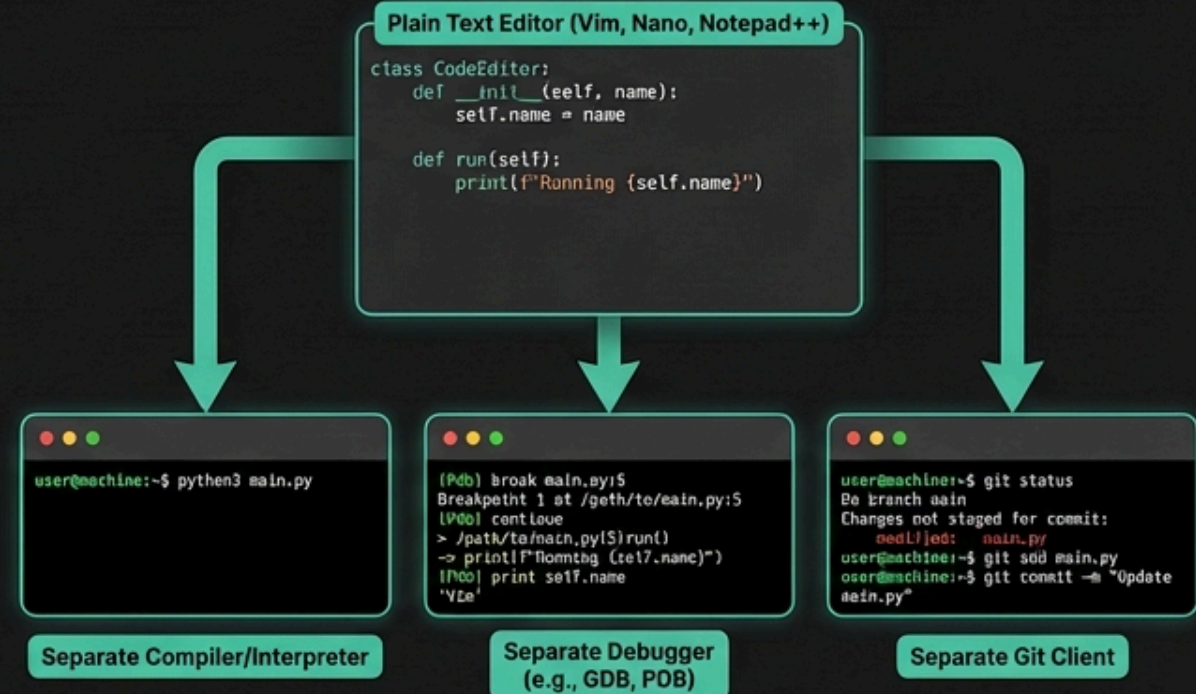
IDE: Integrated Development Environment

All-in-One Application for Streamlined Workflow



Plain Text Editor + Separate Tools

Modular Workflow for Maximum Flexibility



Examples:

- **VS Code** (Web, General, Multi-language)
- **IntelliJ IDEA** (Java, Kotlin, Scala)
- **Eclipse** (Java, C/C++, PHP)
- **PyCharm** (Python, Django)
- **Xcode** (Swift, Objective-C, macOS/iOS)

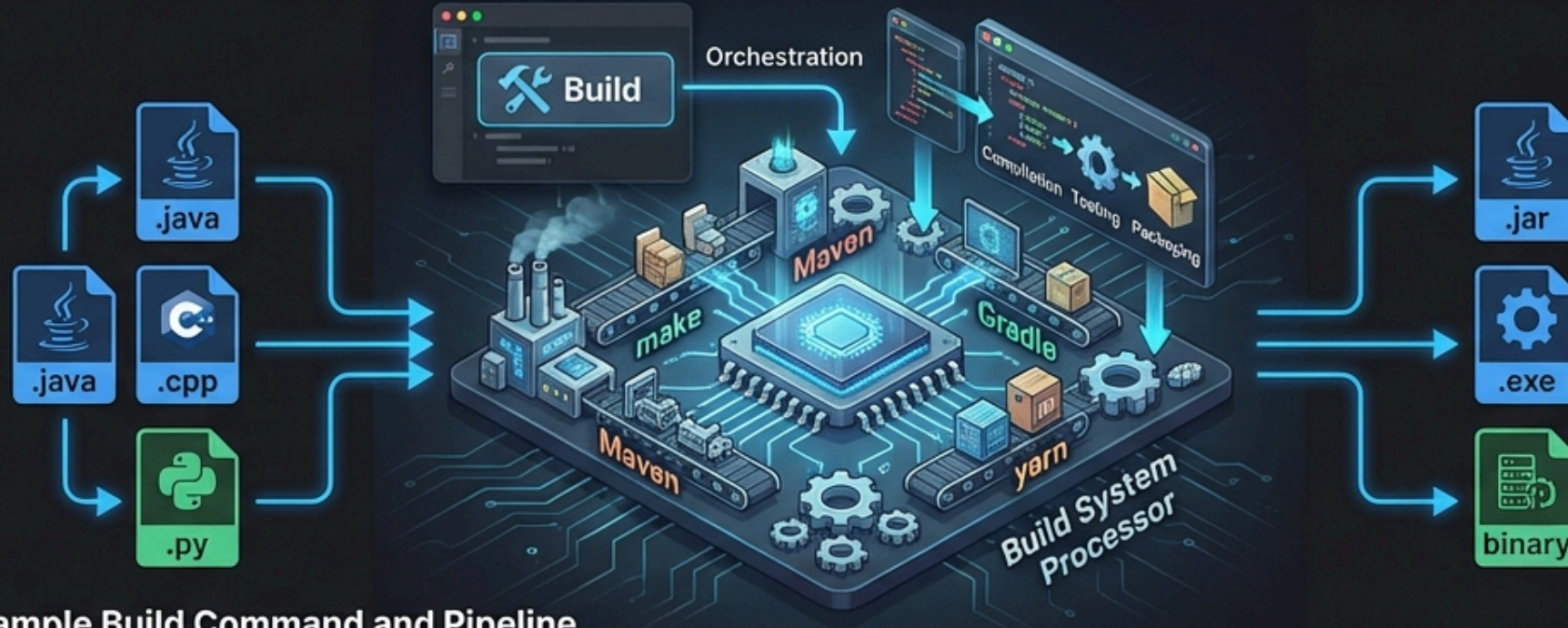


Examples:

- **Vim** (Highly configurable, keyboard-driven)
- **Nano** (Simple, easy to use)
- **Notepad++** (Lightweight, Windows-focused)



Build Tools and Build Systems

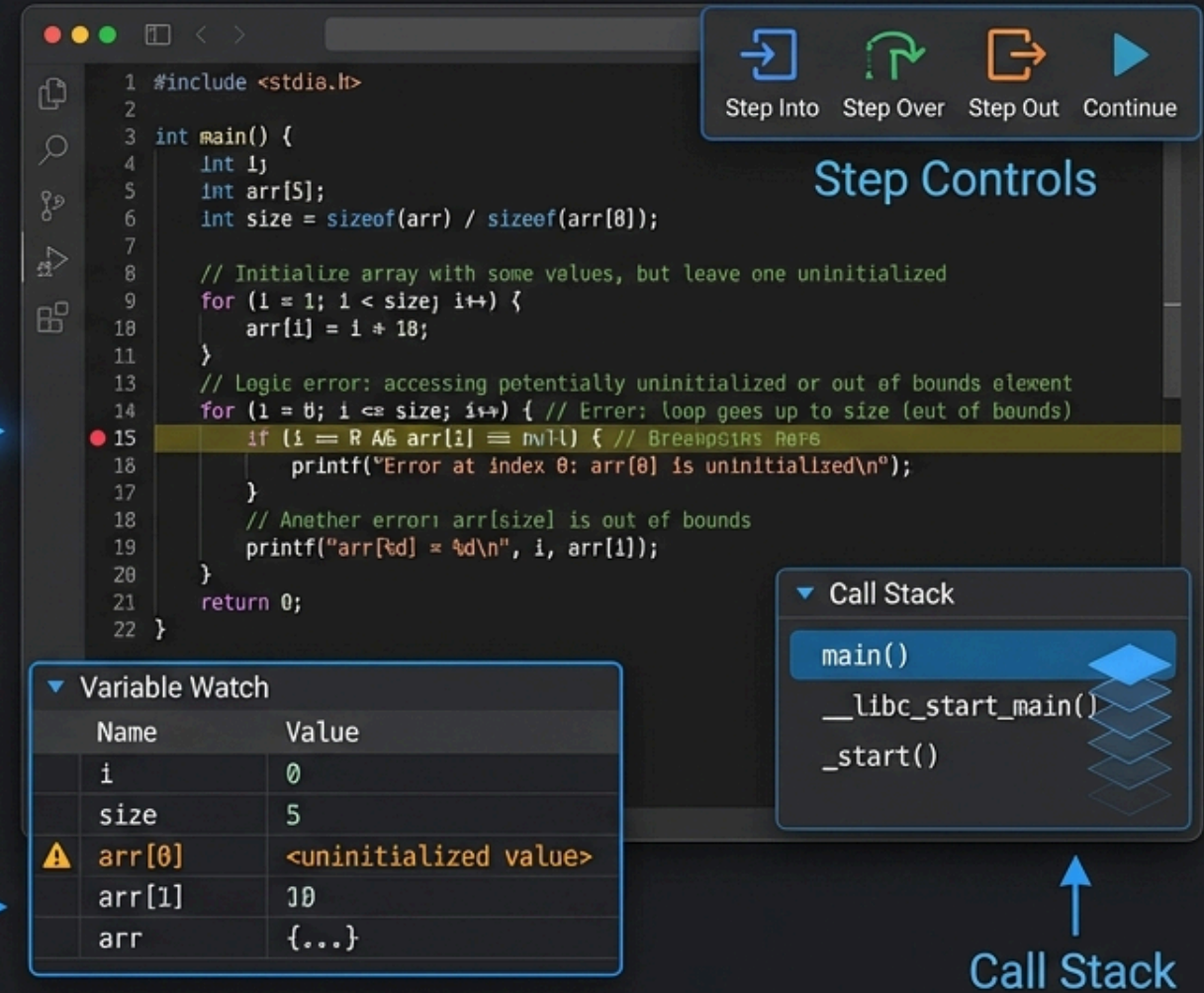


Example Build Command and Pipeline

```
$ mvn clean package
```

```
[INFO] Scanning for projects...
[INFO]
[INFO] Building Java Project Example 1.0-SNAPSHOT
[INFO]
[INFO] --- maven-clean-plugin:3.1.0:clean (default-clean) @ java-project-example ---
[INFO] Deleting /path/to/project/target
[INFO]
[INFO] --- maven-resources-plugin:3.0.2:resources (default-resources) @ java-project-example ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 1 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.8.0:compile (default-compile) @ java-project-example ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 5 source files to /path/to/project/target/classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.1:test (default-test) @ java-project-example ---
[INFO] Running com.example.AppTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.08 s - in com.example.AppTest
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ java-project-example ---
[INFO] Building jar: /path/to/project/target/java-project-example-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.513 s
[INFO] Finished at: 2024-05-15T10:00:00Z
[INFO]
```


Debuggers: Finding Logic Errors



Profilers: Finding Performance Bottlenecks



What is a Profiler?

A profiler measures how often and how long functions run, showing where time is spent in your code.

- ⚙ Measures execution time and frequency.
- ⚙ Identifies code hotspots.
- ⚙ Enables performance optimization.



Tracing Profilers

Records every function call for high accuracy. High overhead.

```
$ perf record -a -g my_app
```

↳ e.g., 100% function_A -> 80% function_B -> ...

Profiler Types



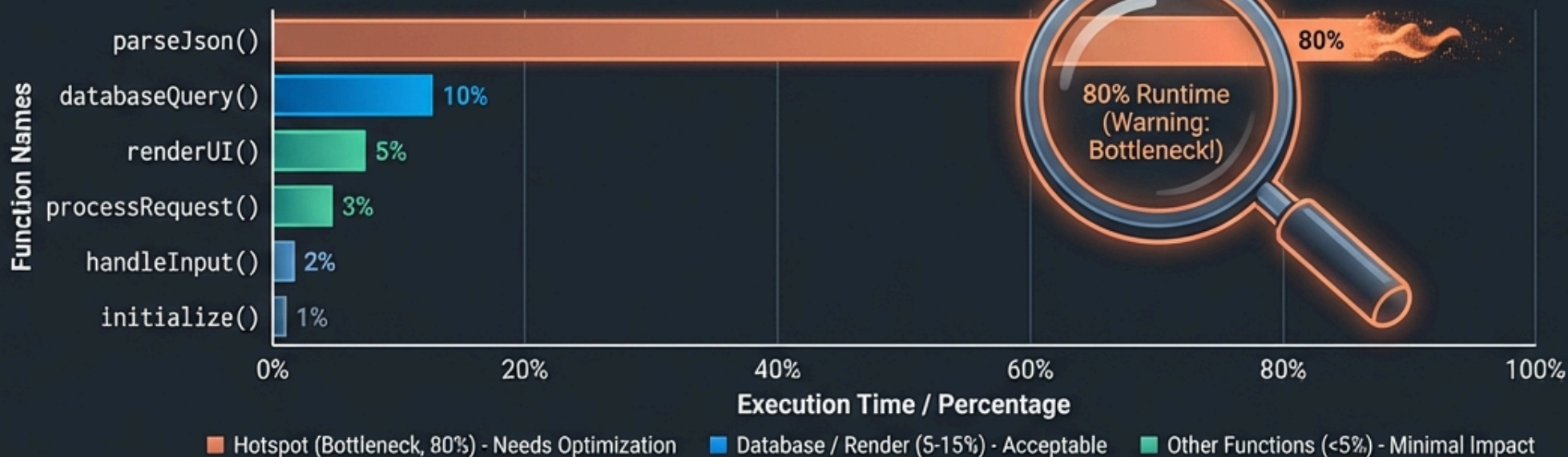
Sampling Profilers

Periodically samples the call stack. Lower overhead.

```
$ perf record -F 99 -a -g my_app
```

↳ e.g., 10s function_A, 5s function_B, ...

Performance Visualization: Identifying the Hotspot



Example: Analysis reveals 80% of runtime is spent in `parseJson()` and not in `databaseQuery()`, guiding optimization efforts.

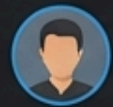
SOURCE CODE MANAGEMENT & VERSION CONTROL



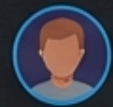
REVERT CHANGES: Version control records history, allowing you to easily revert files or the entire project to any previous state.



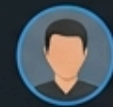
Initial commit
c3d5f9a
2023-10-27 09:00 AM
DevA



Add project structure
a1b2c3d
2023-10-27 11:30 AM
DevB



Update documentation
g9h8i7j
2023-10-28 04:00 PM
DevC



Merge branch 'feature/login'
m4n5o6p
2023-10-30 11:00 AM
DevB



BRANCH & EXPERIMENT: Create isolated branches to work on new features or experiments without affecting the main codebase, then merge back.



Implement login form
f8e7d6c
2023-10-28 02:00 PM
DevA



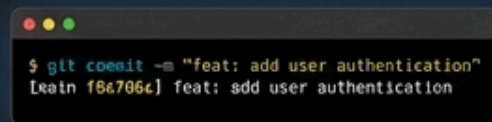
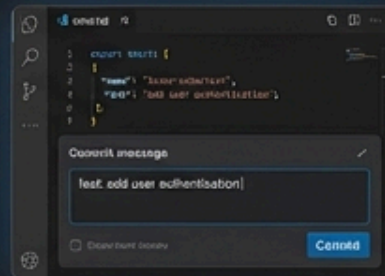
Add validation
e5d4c3b
2023-10-29 10:00 AM
DevA



TRACK OWNERSHIP: Every change is attributed to an author with a timestamp, providing clear accountability and collaboration history.



Distributed Version Control System: Every developer has a full local copy of the repository history.



Why the Command Line Still Matters



- ✓ **SCRIPTABLE & AUTOMATABLE:** Define workflows as reusable scripts.
- ✓ **CRONTAB & CI/CD:** Schedule tasks and integrate into continuous pipelines.
- ✓ **REPRODUCIBILITY:** Ensure consistent execution across environments.

```
#!/bin/bash
# Backup.sh

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
tar -czvf backup_${TIMESTAMP}.tar.gz /data
scp backup_${TIMESTAMP}.tar.gz user@remote:/backups/
echo "Backup completed!"
```

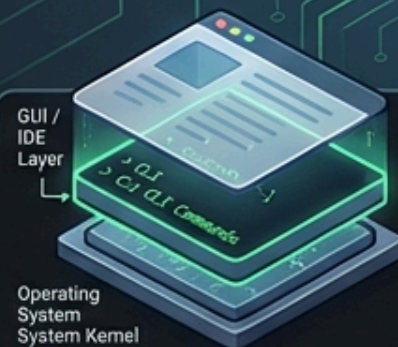
Automation Power



- ✓ **CHAINING TOOLS WITH PIPES:** Combine simple tools for complex tasks.
- ✓ **DATA FLOW:** Process streams of data efficiently.
- ✓ **COMPOSABILITY:** Build powerful workflows from small, focused utilities.

```
curl -s https://api.example.com/data.json | jq '.items[] |
```

Tool Chaining



Many GUI tools including IDEs are thin layers over underlying CLI commands, so knowing the CLI gives deeper control.

- ✓ **UNDERSTANDING THE ABSTRACTION:** See what's happening under the hood.
- ✓ **DEBUGGING:** Diagnose issues obscured by GUI layers.
- ✓ **DIRECT OS INTERFACE:** Access system state and lower-level functions.

Deep Control



- ✓ **WORK OVER SSH:** Securely manage remote servers and cloud instances.
- ✓ **LOW BANDWIDTH:** Efficient for managing headless systems and devices.
- ✓ **ADMINISTRATION:** Essential for system administration, deployment, and troubleshooting.

```
> ssh user@remote-server
Welcome to Ubuntu 22.04 LTS (GNU/Linux 5.12.0-generic x86_64)
Last login: Sat Oct 26 12:36:13 2024 from 192.168.1.100
user@remote-server:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse
   Loaded: loaded (/lib/systemd/system/nginx.service; auto)
   Active: active (running) since Sat 2024-10-26 10:22:22
```

SSH Remote Access

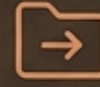
Basic Navigation Commands



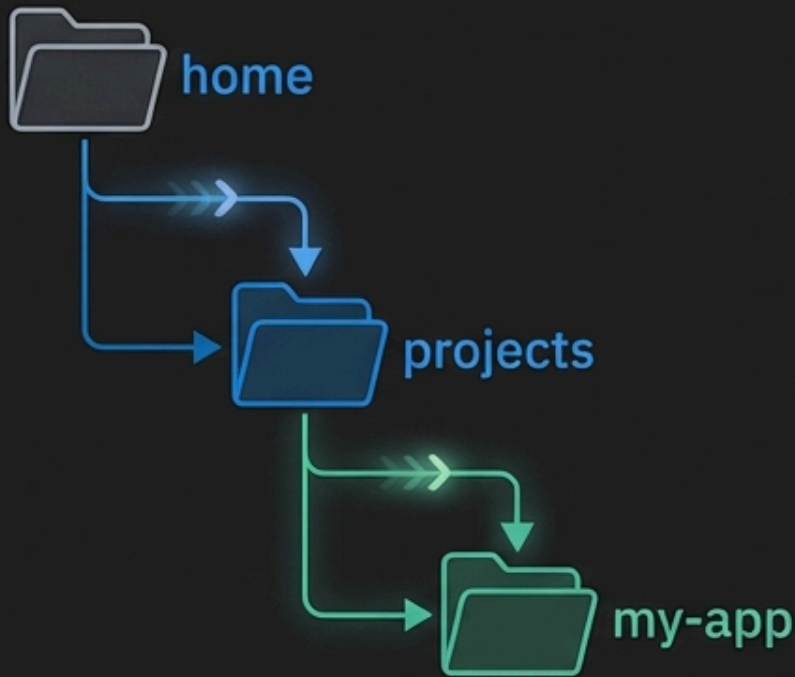
pwd - prints current working directory



ls - lists files and directories



cd - changes directories
(cd path/, cd ..)



home > projects > my-app

```
Terminal - bash

user@machine:~$ pwd
/home
user@machine:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos projects
Desktop Intages
user@machine:~$ cd projects
user@machine:~/projects$ pwd
/home/projects
user@machine:~/projects$ ls
my-app other-project
user@machine:~/projects$ cd my-app
user@machine:~/projects/my-app$ pwd
/home/projects/my-app
user@machine:~/projects/my-app$ cd ..
user@machine:~/projects$ pwd
/home/projects
```

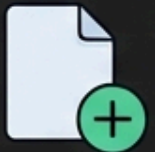

Working with Files and Directories



mkdir

Syntax: `mkdir <directory_name>`

Creates a new directory.



touch

Syntax: `touch <filename>`

Creates an empty file.



cp

Syntax: `cp <source_file> <destination>`

Copies files or directories.



mv

Syntax: `mv <source_file> <destination/new_name>`

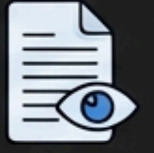
Moves or renames files or directories.



rm

Syntax: `rm <filename>`

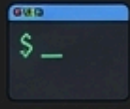
Removes a file (use with caution).



cat

Syntax: `cat <filename>`

Prints file contents to standard output.



grep

Syntax: `grep <pattern> <filename>`

Searches for lines containing a specific pattern.





Redirection and Pipes

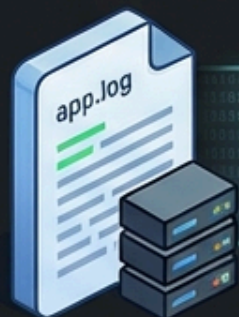


```
grep ERROR app.log | sort | uniq -c > error-summary.txt
```

Example: Filters, aggregates, and saves error counts.

Input File

app.log



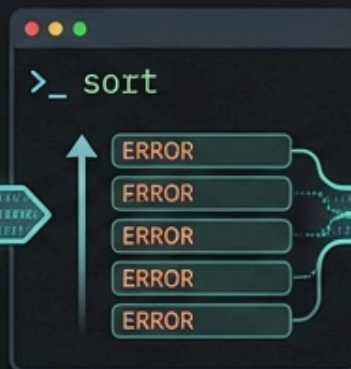
Source data,
e.g., log file

1. grep ERROR



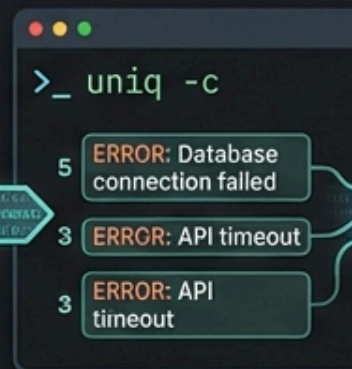
Filters data:
Selects lines
containing 'ERROR'

2. sort



Sorts data:
Arranges lines
alphabetically

3. uniq -c



Aggregates data:
Counts unique lines

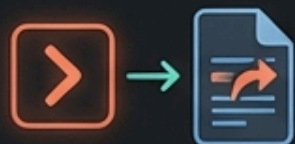
Output File

error-summary.txt

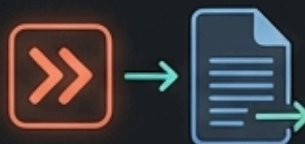


Redirects standard
output to a file
(overwriting contents)

Redirection Operators



> file: Redirects standard
output, overwriting existing
file content.



>> file: Redirects standard
output, appending to
existing file content.

Getting Help from the Shell

Understand how to access documentation and usage information directly from the command line for efficient development.

```
Terminal - zsh
user@host:~$ man ls
LS(1)                                     User Commands                                LS(1)

NAME
  ls - list directory contents

SYNOPSIS
  ls [OPTION]... [FILE]...

DESCRIPTION
  List information about the FILES (the current directory by default). Sort
  entries alphabetically if none of -cftuvSUX nor --sort is specified.

  -a, --all                do not ignore entries starting with .
  -l                       use a long listing format
  -h, --human-readable    with -l and/or -s, print human readable sizes (e.g., 1K
                          234M 2G)
  --help                  display this help and exit

EXAMPLES
  ls -la
  List all files, including hidden ones, in long format.

  ls -lh /var/log
  List contents of /var/log in long format with human-readable sizes.
```

Detailed
Documentation
& Manuals

Quick Usage
Information &
Command Lists

```
Terminal - zsh
user@host:~$ git --help
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
  [--exec-path<gitpath>] [--html-path] [--man-path]
  [--info-path] [-p|--paginate|--no-pager] [--no-replace-
  objects] [-b] [--git-dir<gitpath>] [--work-tree<workpath>]
  [--bare] [--namespace<name>] [--config<name>=<value>]
  <command> [<args>]

These are common Git commands used in various situations:

start a working area
  clone Clone a repository into a new directory
  init Create an empty Git repository or reinitialize an existing one

work on the current change
  add Add file contents to the index
  mv Move or rename a file, a directory, or a symlink
  restore Restore working tree files
  rm Remove files from the working tree and from the index

examine the history and state
  diff Show changes between commits, commit and working tree, etc
  log Show commit logs
  show Show various types of objects
  status Show the working tree status

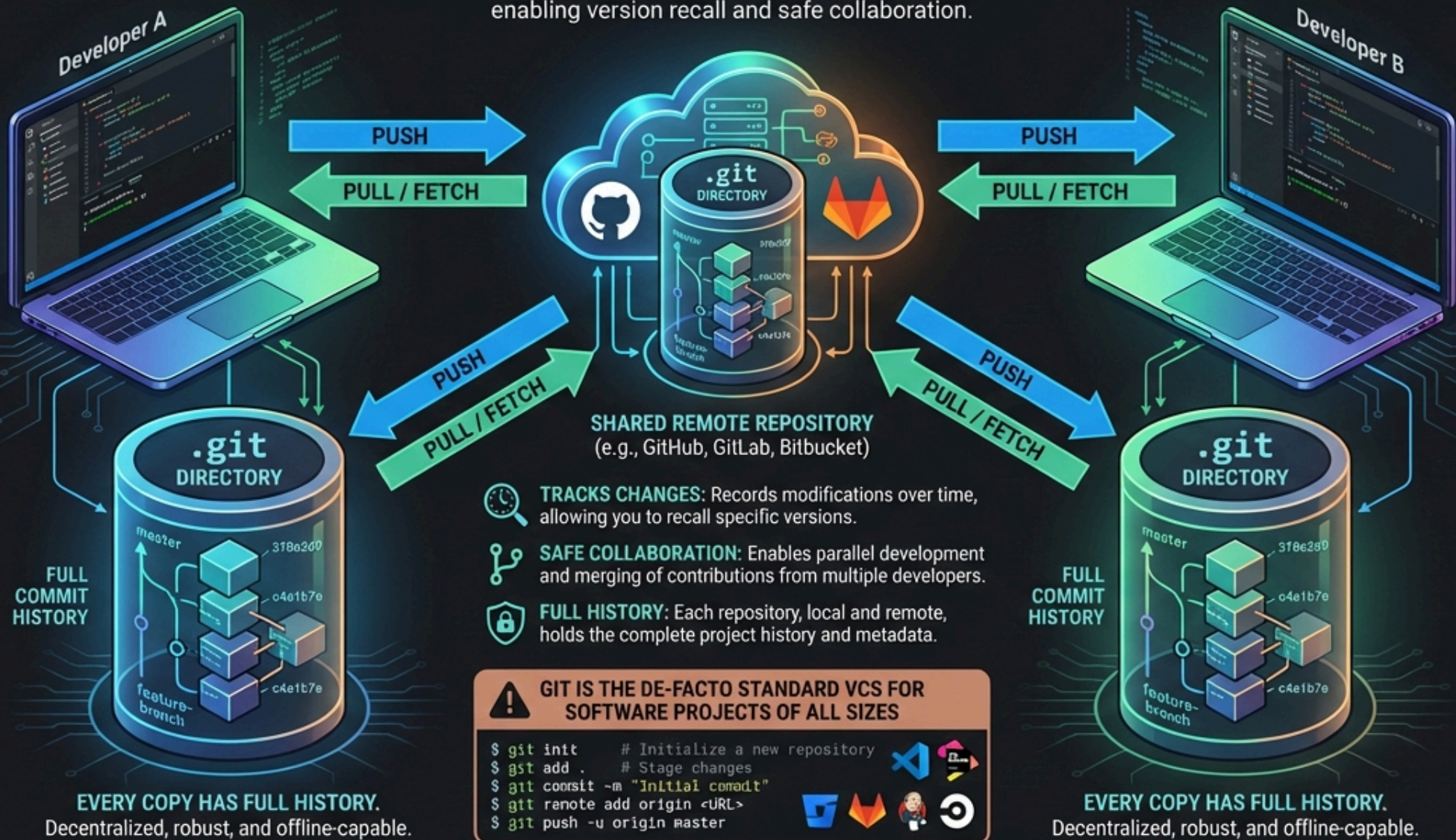
grow, mark and tweak your common history
  branch List, create, or delete branches
  commit Record changes to the repository
  merge Join two or more development histories together
  rebase Reapply commits on top of another base tip
  reset Reset current HEAD to the specified state
  switch Switch branches
  tag Create, list, delete or verify a tag object signed with GPG

collaborate
  fetch Download objects and refs from another repository
  pull Fetch and integrate with another repository or a local branch
  push Update remote refs along with associated objects
```

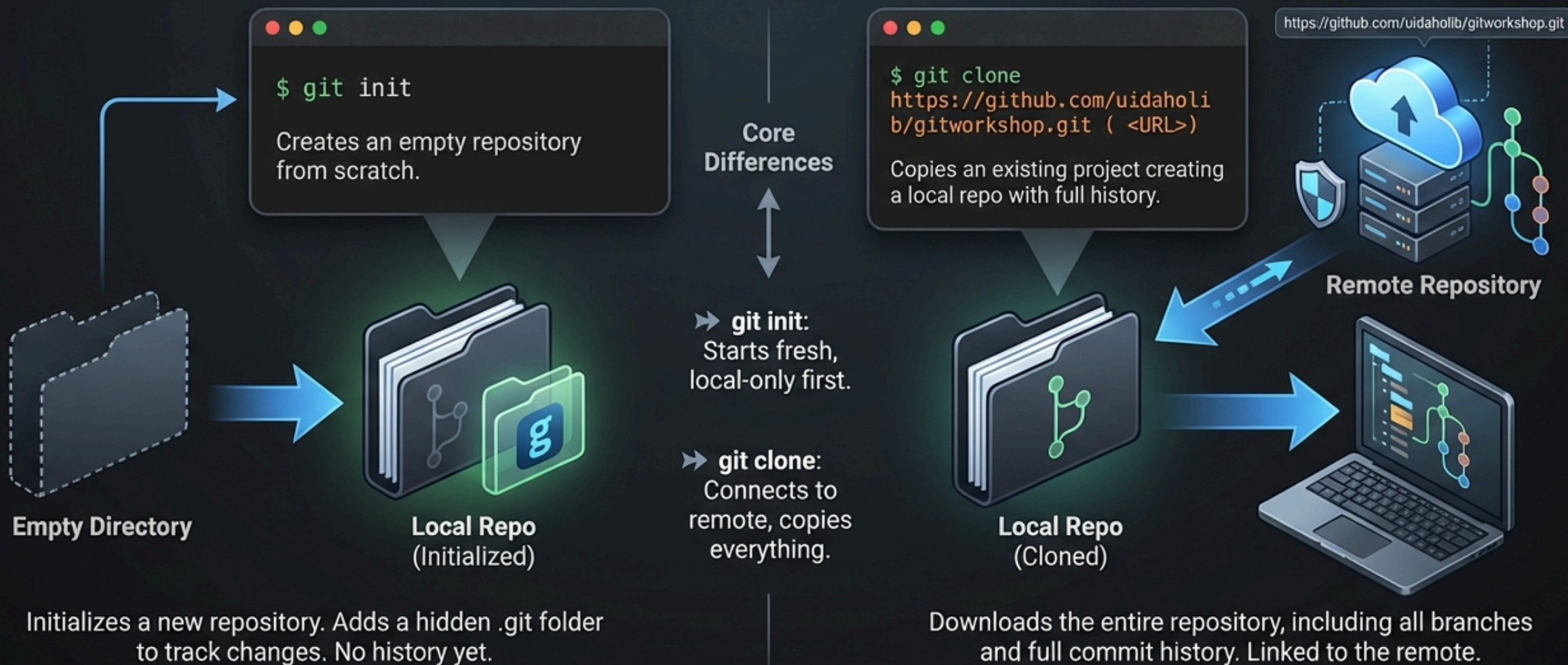



What Is Git? Quick Overview

Distributed Version Control System (DVCS) tracking changes to files, enabling version recall and safe collaboration.

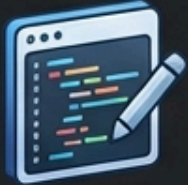


Cloning or Creating a Repository



The Git Three Areas: Working, Staging, Repository

Working Directory



Editable files

```
1 # main.py
2 def calculate_total(items):
3     total = 0
4     for item in items:
5         total += item.price
6     # EDIT: Fixed bug here
7     return total
```

- 📄 Current state of files
- 📁 Where changes and edits occur
- 🔄 Changes are not yet tracked by Git

Staging Area / Index



Records what will go into the next commit

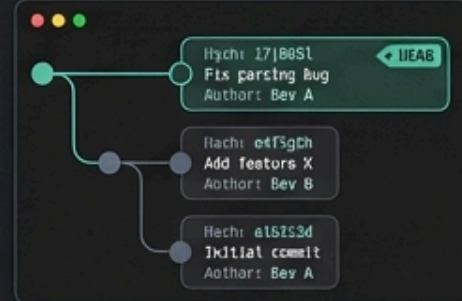
```
1 # main.py (staged) ✓
2 def calculate_total(items):
3     total = 0
4     for item in items:
5         total += item.price
6     # EDIT: Fixed bug here
7     return total
8
9 # [✓] Ready for commit
```

- 📄 Prepare changes for the next snapshot
- 📄 Selectively choose files to commit
- 🔄 Also known as the 'index'

Repository



Stores committed snapshots



- 📄 Permanent history of the project
- 📄 Contains all committed snapshots
- 📄 Safe storage, distributed across clones

git add

```
git add main.py
```

git commit

```
git commit -m "Fix parsing bug"
```

Example Workflow



Edit File

Edit `main.py` to fix a bug.

```
# Make changes in the editor
```



Stage Changes

Run `git add main.py` to move changes from Working to Staging.

```
$ git add main.py
```



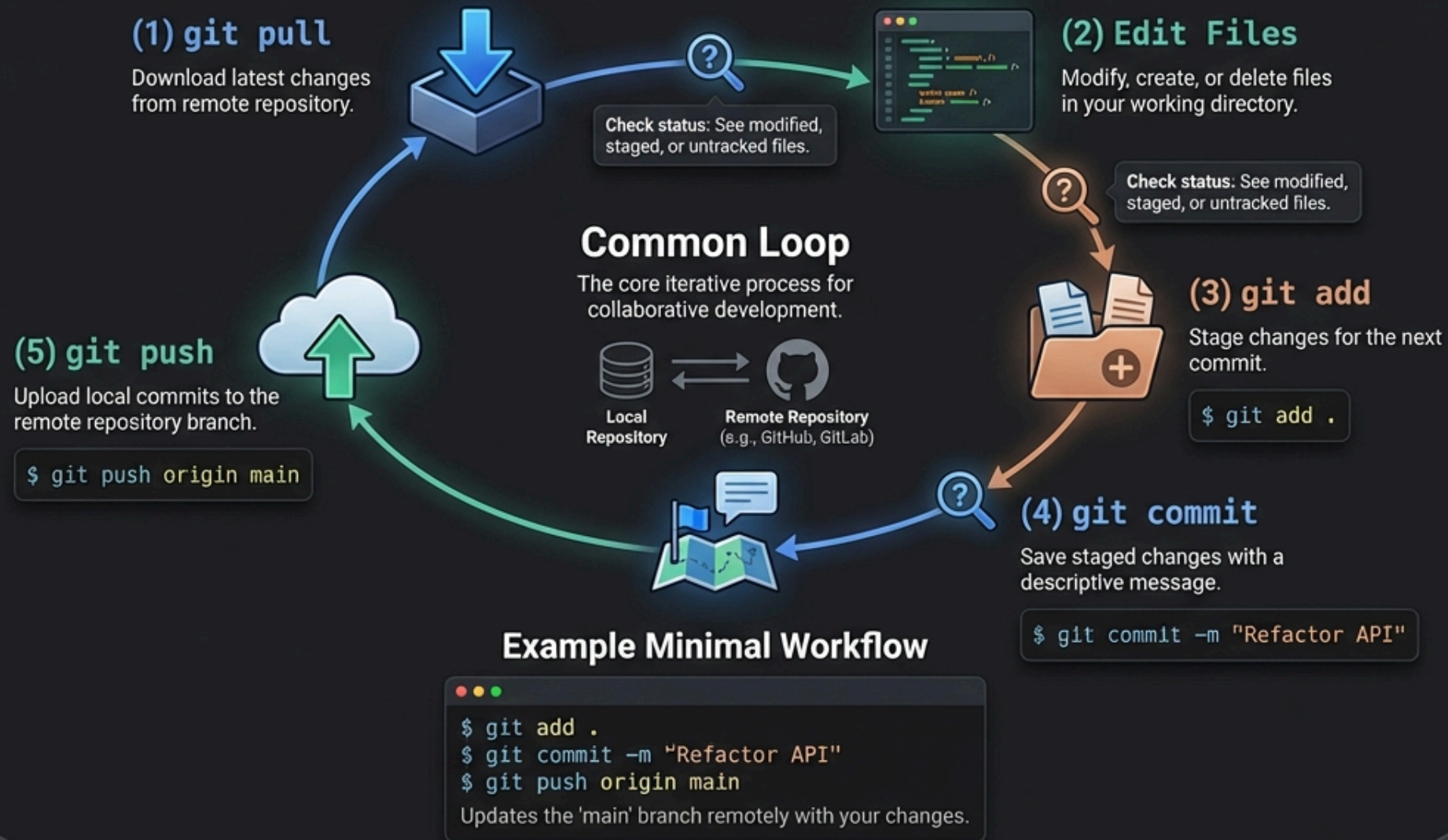
Commit Snapshot

Run `git commit -m "Fix parsing bug"` to record the snapshot in the Repository.

```
$ git commit -m "Fix parsing bug"
[main 1718051] Fix parsing bug
1 file changed, 1 insertion(+), 1 deletion(-)
```

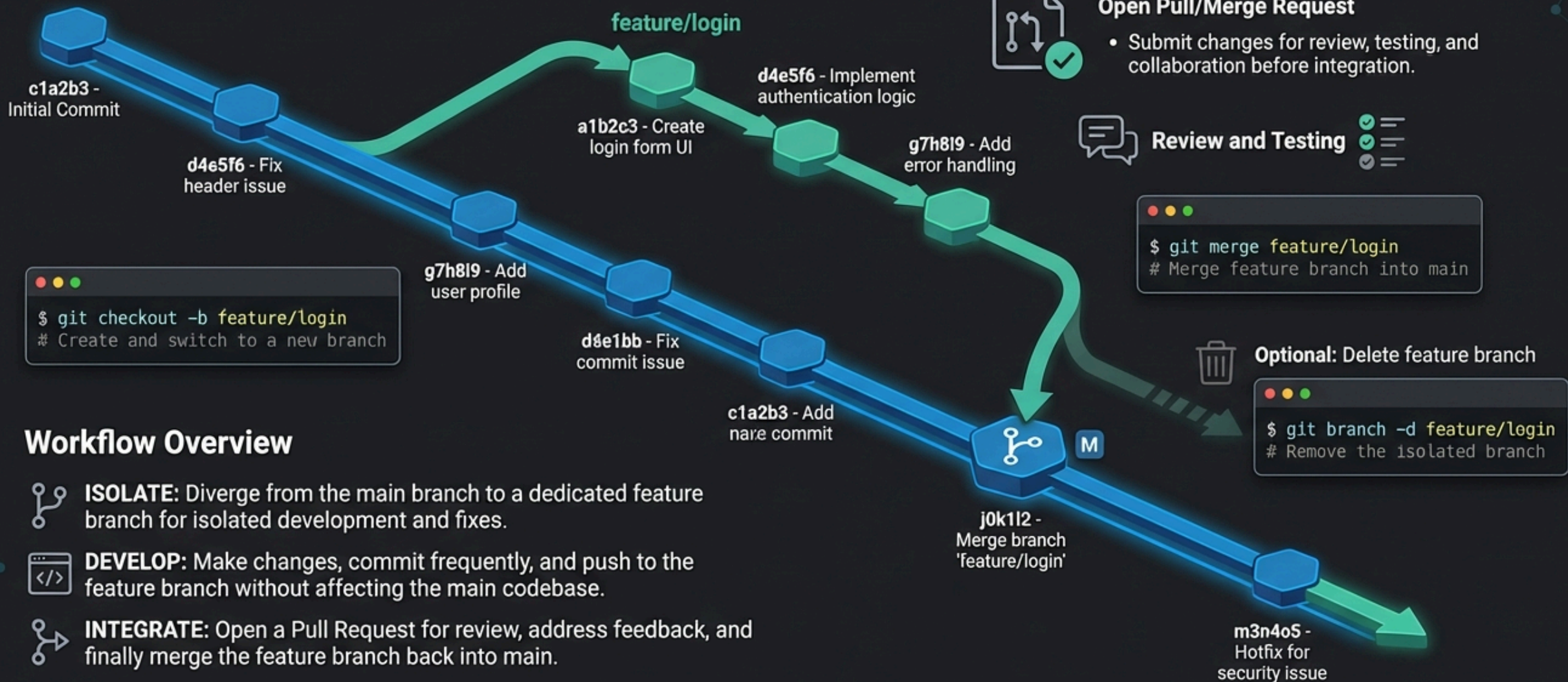


Basic Local → Remote Workflow

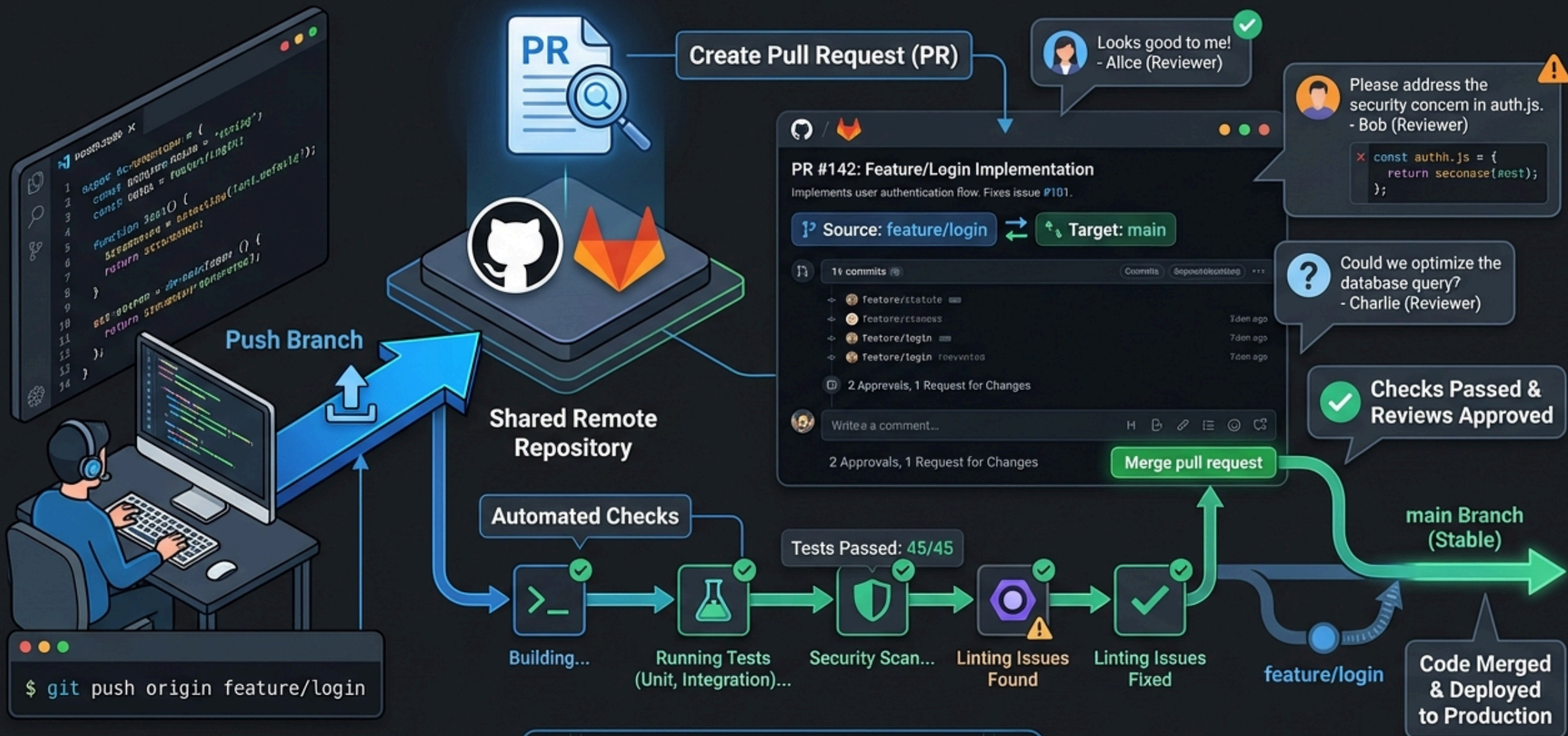


Branching and Feature Development

main branch



Collaborative Workflow & Pull Requests



Inspecting History and Changes

Understand Git history and review modifications with powerful commands. **git log** reveals commit lineage with metadata, while **git diff** compares variations across the working directory, staging area, and commits for precise review before committing.

```
user@developer-machine:~/project$ git log --oneline --graph --decorate

* 6a1b3c8 (HEAD -> main) feat: Implement user authentication and profile page
| Author: Nabajyoti Medhi <nabajyoti@example.com>
| Date: Mon Oct 23 14:30:22 2023 +0530
|/
* 4d2e5f9 fix: Resolve issue with database connection pooling
| Author: Sara Johnson <sara@example.com>
| Date: Fri Oct 20 11:15:45 2023 +0530
|/
* 1g8h9i0 (origin/main) docs: Update README with project setup instructions
| Author: Nabajyoti Medhi <nabajyoti@example.com>
| Date: Wed Oct 18 09:50:10 2023 +0530
|/
* 7j8k1l2 chore: Refactor utility functions for better readability
| Author: David Lee <david@example.com>
| Date: Mon Oct 16 16:20:33 2023 +0530
|/
* * 3m4n5o6 init: Initial project structure and configuration
| Author: Nabajyoti Medhi <nabajyoti@example.com>
| Date: Thu Oct 12 10:05:00 2023 +0530
```

 **git log** displays commit history with hashes, authors, timestamps, and messages, providing a chronological record of changes.

↔ **git diff** compares changes between working directory, staging area, and commits, helping you review before committing. Examples: **git diff HEAD~1..HEAD** (last commit changes), or **git diff origin/main** (local vs remote).

```
user@developer-machine:~/project$ git diff HEAD~1..HEAD

diff --git a/src/auth.js b/src/auth.js
index e63de29..495f8g7 100644
--- a/src/auth.js
+++ b/src/auth.js
@@ -10,14 +10,10 @@
 // New function for user login
- function loginUser(username, password) {
-   console.log('Logging in $(username)');
-   // TODO: Implement actual login logic
- }
+ async function loginUser(username, password) {
+   try {
+     const user = await database.findUser(username);
+     if (user && await bcrypt.compare(password, user.passwordHash)) {
+       console.log('User $(username) logged in successfully.');
```

↔ **git diff** output with green (+) and red (-) lines showing added and removed code, facilitating detailed code review.



What Is a Shell?

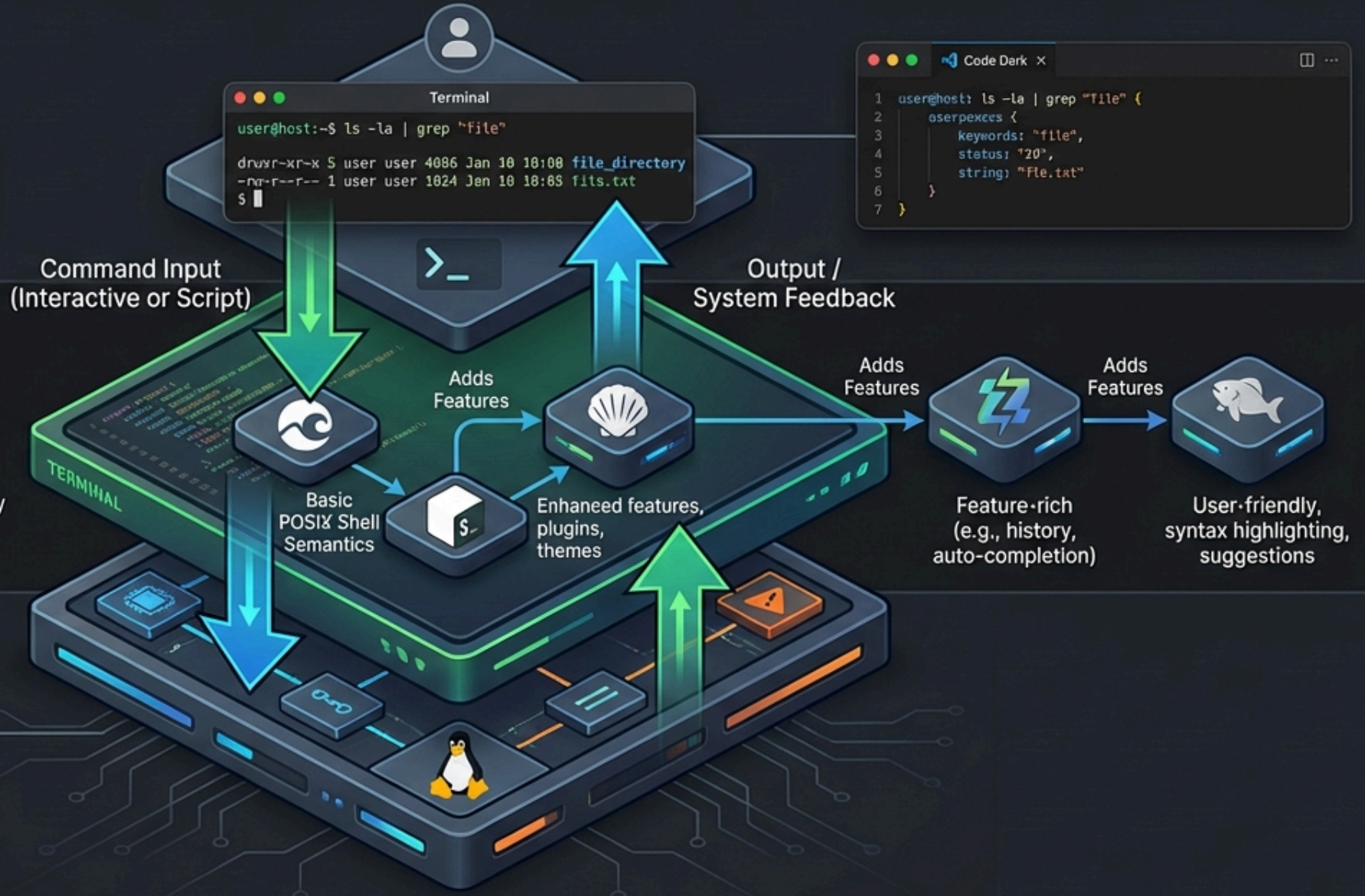
 **USER**
(Interacting via Terminal)

```
user@host:~$ ls -li & /
-- FudtT---r--r-- "user | rep loerx "File?")
$
```

 **SHELL LAYER**
(Command Language Interpreter)

The Shell interprets commands, letting you interact with the OS by typing commands, either interactively or via scripts. It is typically the default command interpreter for user terminals on GNU/Linux systems.

 **OS KERNEL**
(e.g., Linux)



Bash: The Bourne Again Shell



Variables

```
$VAR="value"
echo "The value is: $VAR"
# Output: The value is: value
```

Store and retrieve data using variables (\$). Variables can hold strings, numbers, and arrays.



Functions

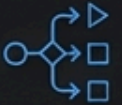
```
function greet_user() {
    local name=$1
    echo "Hello, $name! Welcome back."
}
greet_user "Alice"
```

Encapsulate reusable logic in functions for better code organization and modularity.

bash_

Bourne Again Shell: Widely available command interpreter & default on GNU/Linux. Automate non-trivial programs.

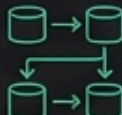
Control Flow



```
if [ $COUNT -gt 10 ]; then
    echo "Count is high"
else
    for i in {1..5}; do
        echo "Iteration: $i"
    done
fi
```

Implement decision-making with `if`, `case` and loops with `for`, `while`, `until`.

I/O Redirection

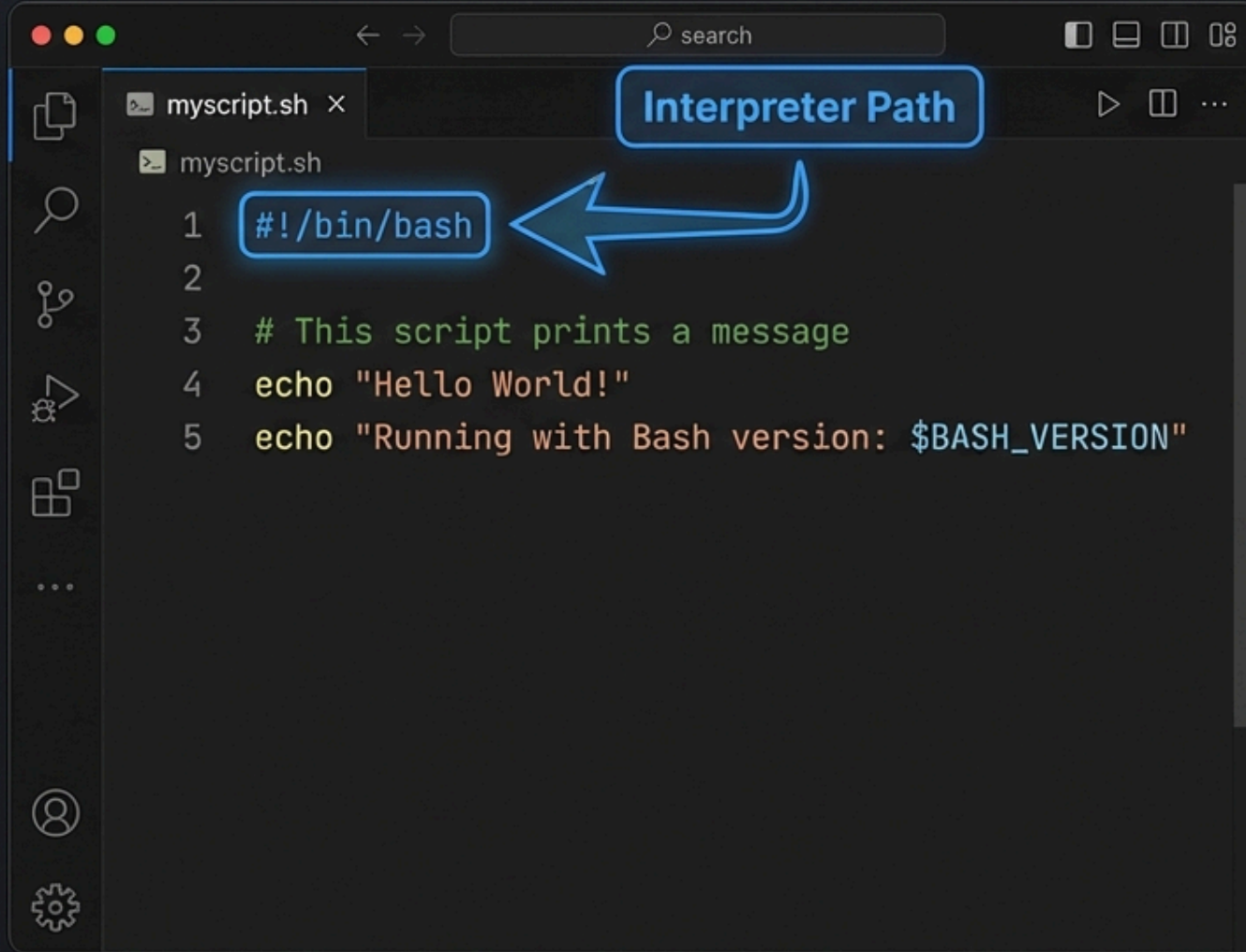


```
grep "ERROR" logfile.txt > errors_found.log
cat file1.txt | sort | uniq >> combined_sorted.txt

./script.sh 2> error_log.txt
```

Redirect standard input (<), output (>), and error (2>) streams, and chain commands with pipes (|).

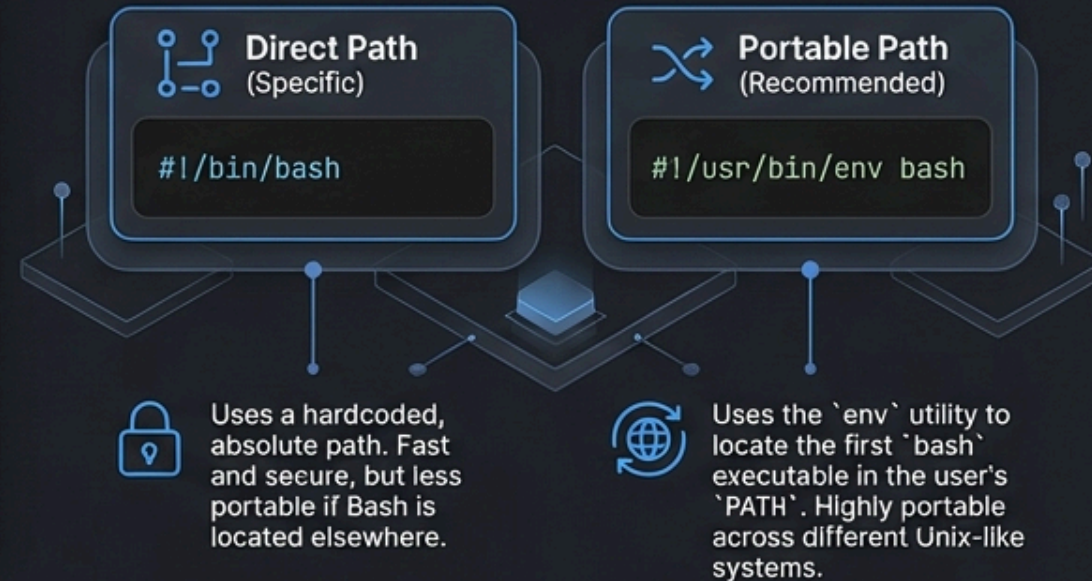
The Shebang Line (#!)



```
1 #!/bin/bash
2
3 # This script prints a message
4 echo "Hello World!"
5 echo "Running with Bash version: $BASH_VERSION"
```

- **Purpose:** Tells the Operating System which interpreter program to use for executing the script.
- **Placement:** Must be the absolute first line (starts with `#!`).
- **Functionality:** The OS reads the path after `#!` and executes that program with the script as an argument.
- **Without Shebang:** The script runs with the user's current default shell, which may not be Bash (e.g., might be Zsh, Dash), potentially causing errors or unexpected behavior. ⚠️

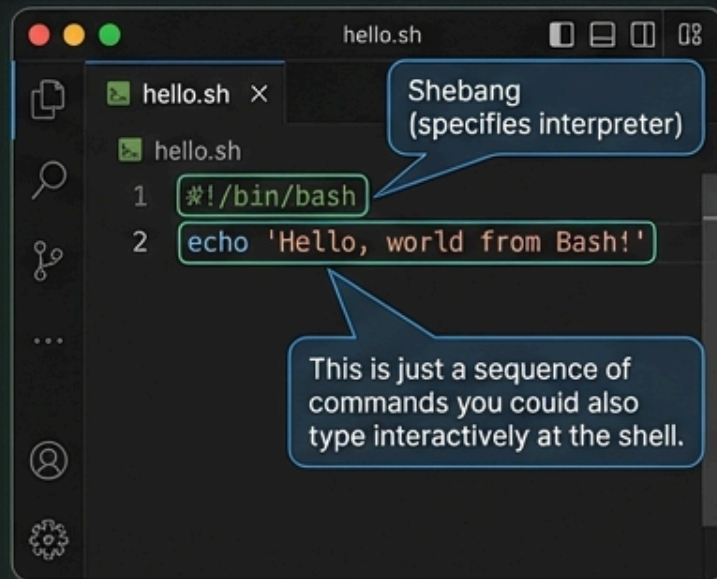
Shebang Comparison



Writing Your First Bash Script

Learn the fundamental steps to create, permission, and execute your first script.

1. Create a file (e.g., hello.sh)



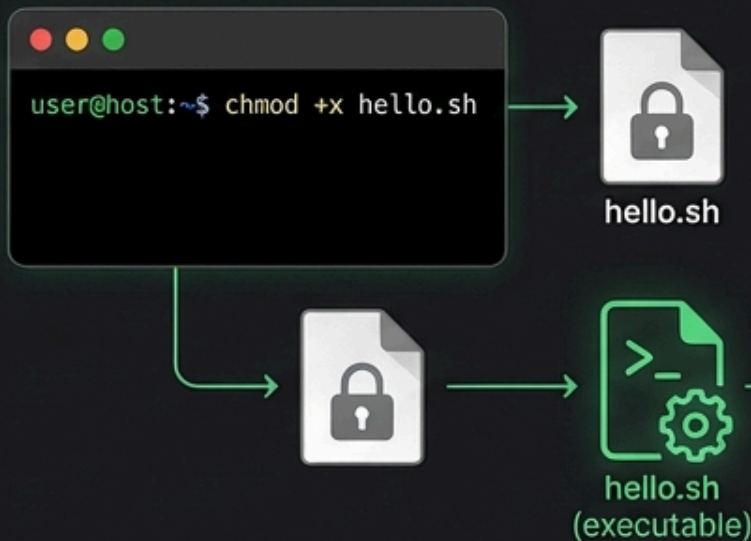
```
hello.sh
1 #!/bin/bash
2 echo 'Hello, world from Bash!'
```

Shebang (specifies interpreter)

This is just a sequence of commands you could also type interactively at the shell.

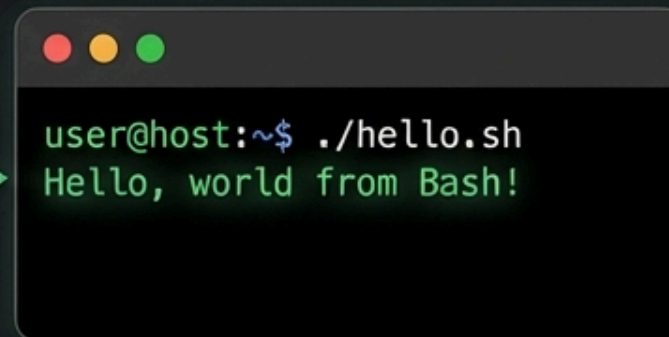
>_ Use a text editor to create the file and add the script content.

2. Make it executable (chmod)



>_ Grants execute permission to the file, allowing it to run as a program.

3. Run it (./hello.sh)

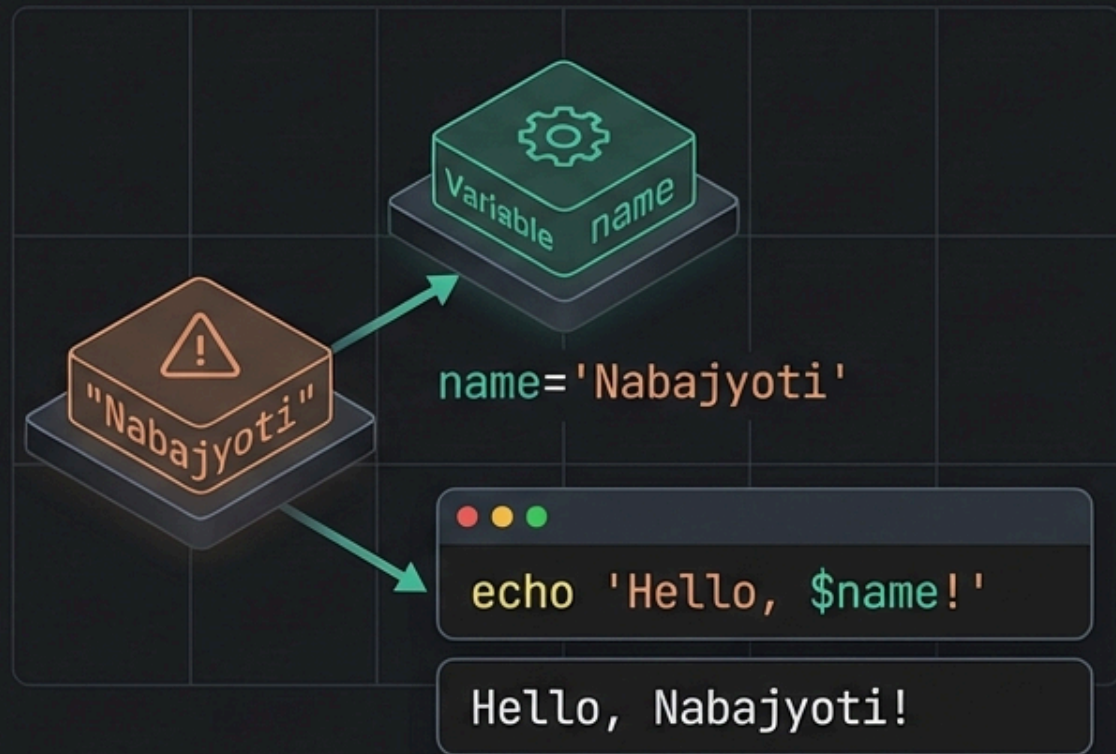


```
user@host:~$ ./hello.sh
Hello, world from Bash!
```

>_ Executes the script using its relative path. The `./` is necessary because the current directory is usually not in your PATH.

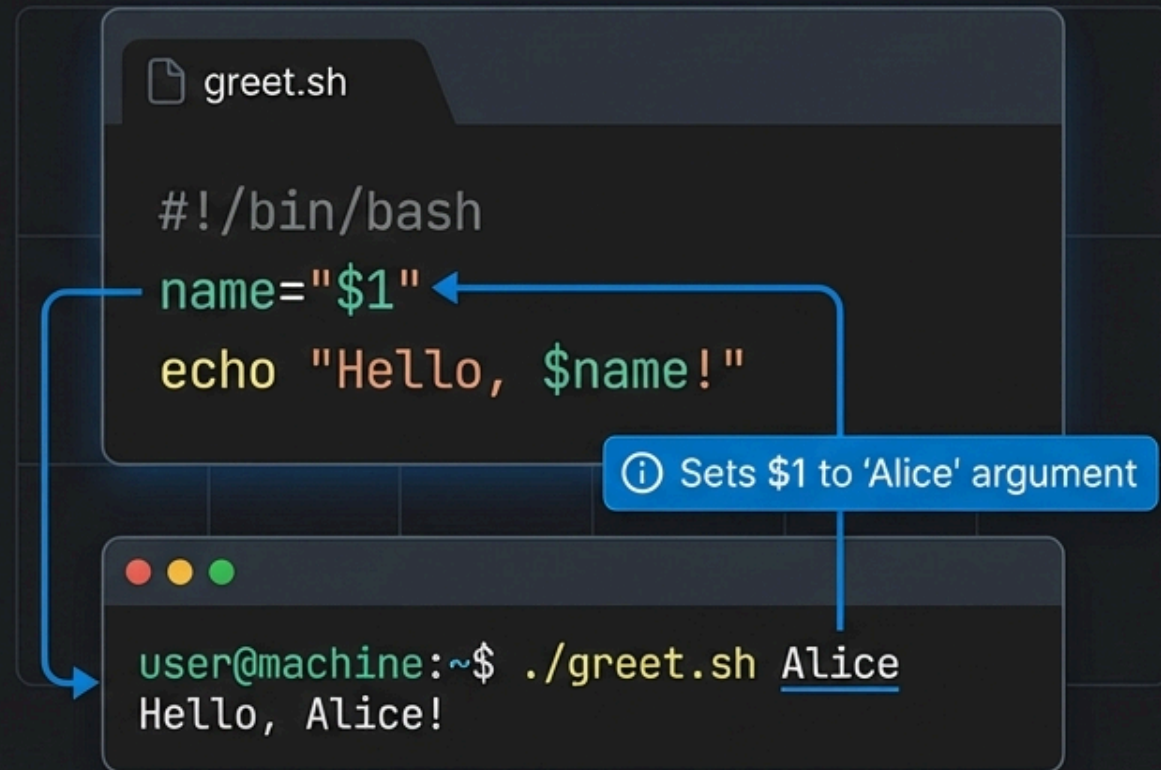
Variables and Script Arguments

Variables in Bash



Store values in variables like `name='Nabajyoti'` then use them via `$name` inside commands.

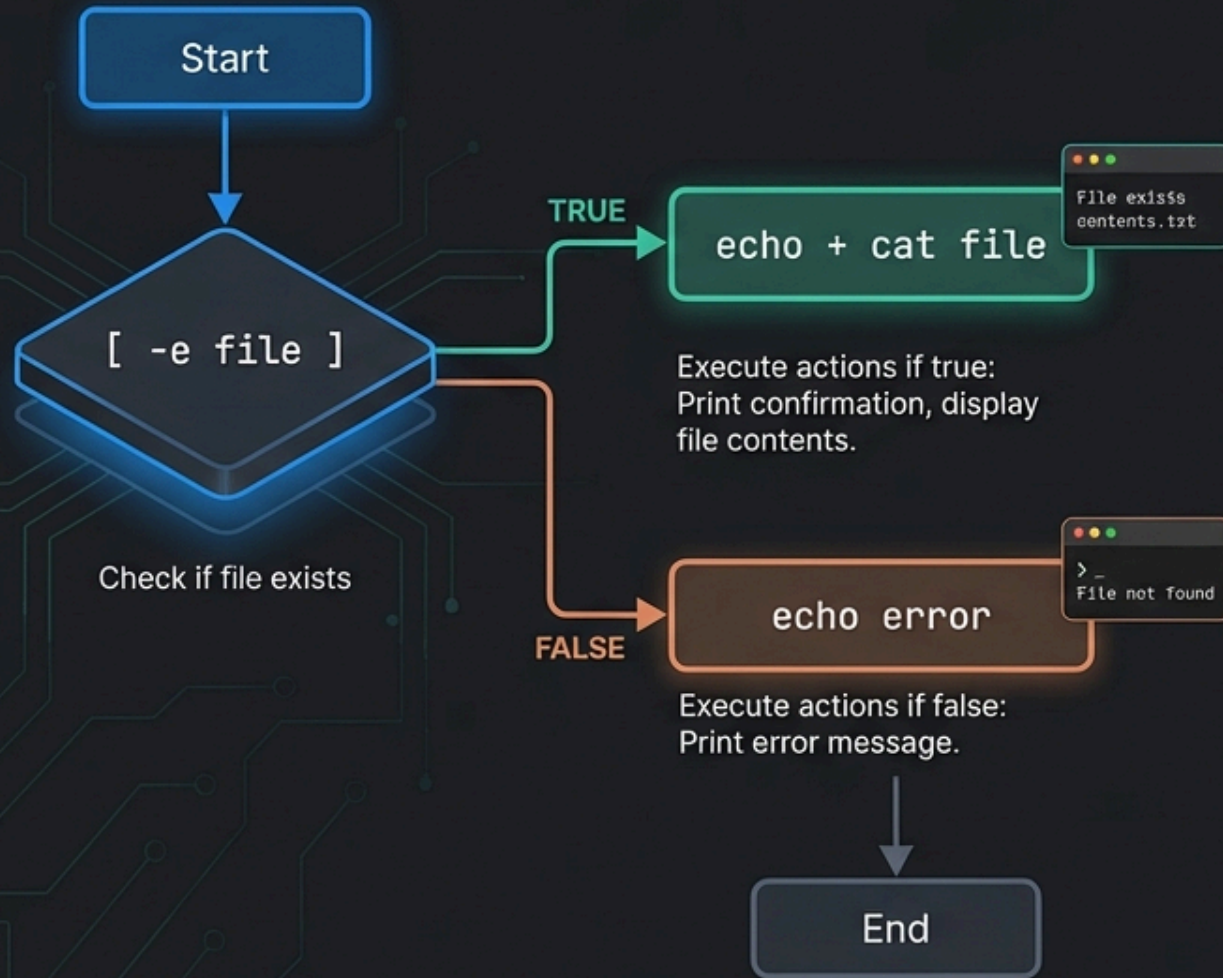
Script Arguments (Positional Parameters)



Positional parameters like `$1`, `$2` refer to arguments passed to the script: `./greet.sh Alice` sets `$1` to Alice.

Conditionals in Bash

Executing Code Blocks Based on Conditions. Focus on File Existence Check.



```
#!/bin/bash

# Check if a file exists before reading it
if [ -e "$1" ]; then
    echo "File exists"
    cat "$1"
else
    echo "File not found"
fi
```

Common Tests

-  **String Comparison**
Test for equality (=), inequality (!=), or if a string is empty (-z) or non-empty (-n).
e.g., ["\$str1" = "\$str2"]
-  **Numeric Comparison**
Compare numbers using operators like -eq (equal), -ne (not equal), -gt (greater than), -lt (less than), -ge (greater than or equal), -le (less than or equal).
e.g., ["\$num" -gt 10]
-  **File Tests**
Check file attributes and existence. -e (exists), -d (is directory), -f (is regular file), -r (is readable), -w (is writable), -x (is executable).
e.g., [-d "/var/log"]

Loops for Repetition

FOR Loops: Iterate over Lists

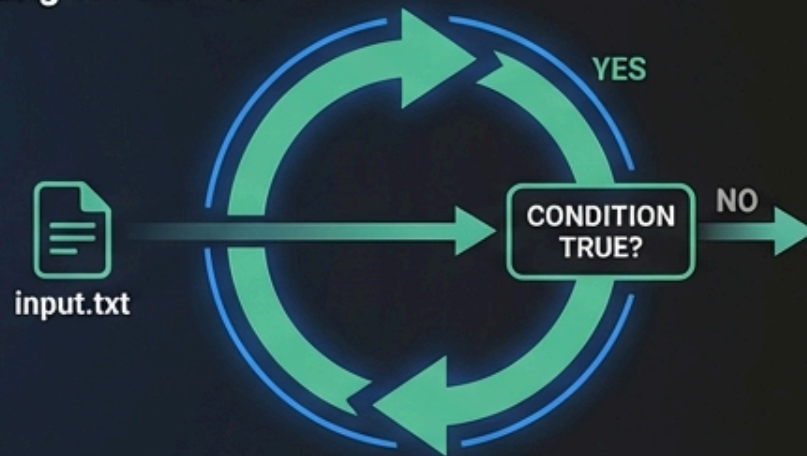
- Iterates through a pre-defined list of items (e.g., files, strings, numbers).
- Executes the block of code once for each item.



```
Bash
# Example: Process all .log files
for f in *.log; do
    echo "Processing $f"
    grep "ERROR" "$f"
done
```

WHILE Loops: Repeat while Condition is True

- Repeats the code block as long as a specified condition remains true.
- Often used for reading streams, processing input, or waiting for events.



```
Bash
# Example: Read lines from a file
while read line; do
    echo "$line"
done < input.txt
```


Functions and Reuse

Local Variables:

Scope limited to the function, preventing global namespace pollution.

Return Exit Codes:

Communicate success (0) or failure (non-zero) to the caller.

Function Definition

```
#!/bin/bash

# Encapsulate reusable logic
function backup() {
    # Local variables
    local src="$1"
    local dest="$2"

    # Perform backup using positional parameters
    cp -r "$src" "$dest"

    # Check exit status of the last command
    if [ $? -eq 0 ]; then
        echo "Successfully backed up $src to $dest"
        return 0 # Success exit code
    else
        echo "Error: Failed to backup $src to $dest" >&2
        return 1 # Failure exit code
    fi
}
```

Parameters \$1, \$2
Access passed arguments within the function.

Parameter Flow

Exit Code Check
Other parts of the script check the function's result.

Function Call & Usage

```
# Main script execution

# Define source and destination directories
SOURCE_DIR="src/"
BACKUP_DIR="backup/"

# Call the function with parameters
echo "Starting backup process..."
backup "$SOURCE_DIR" "$BACKUP_DIR"

# Check the function's exit code
if [ $? -eq 0 ]; then
    echo "Backup completed successfully!"
else
    echo "Backup failed! Please check logs."
fi
```

- **Encapsulation:** Group related logic and data, reducing complexity.
- **Reusability:** Define once, call multiple times with different parameters.
- **Structure & Readability:** Break down large scripts into manageable, named units.

Complete Example Script: functions_example.sh

```
#!/bin/bash

# --- Function Definition ---
function backup() {
    local src="$1"
    local dest="$2"

    # Ensure destination directory exists
    mkdir -p "$dest"
    echo "Backing up '$src' to '$dest'..."
    cp -r "$src" "$dest"
    if [ $? -eq 0 ]; then
        echo "Successfully backed up $src to $dest"
        return 0
    else
        echo "Error: Failed to backup $src to $dest" >&2
        return 1
    fi
}

# --- Main Script Execution ---
echo "Script started."

# Call with directory arguments
backup src/ backup/

# Check exit code of the backup function call
if [ $? -eq 0 ]; then
    echo "Critical Error: Backup operation failed. Exiting."
    exit 1
fi

echo "Script finished successfully."
```

- **Parameters:** Pass dynamic values into functions for flexibility.
- **Local Variables:** Prevent naming conflicts and scope issues.
- **Exit Codes:** Functions return status codes for error handling.

Practical Script: Project Backup Utility

Real-world automation of project backups using variables, command substitution, directories, and the `cp -r` command.



```
TERMINAL: project_backup.sh
#!/bin/bash

# Define source and destination directories
# Uses command-line argument $1 for flexibility
src="$1"

# Generate timestamp for dynamic folder naming
# Command substitution $(date +%F) creates YYYY-MM-DD string
dest="$HOME/backups/$(date +%F)"

# Create the destination directory with its parents if needed
# mkdir -p ensures the path exists, avoiding errors
mkdir -p "$dest"

# Copy the project directory to the timestamped backup location
# cp -r recursively copies the directory and its contents
cp -r "$src" "$dest"

# Provide user feedback
# echo displays the success message with the backup path
echo "Backup complete: $dest"
```

- ≡ **Variables** (`\$src`, `\$dest`): Store paths dynamically for flexibility.
- ⚡ **Command Substitution** (`\$(date ...)`) : Automates dynamic, date-based naming for organized backups.
- 📁 **Directories** (`mkdir -p`): Ensures destination paths exist, preventing errors.
- 📁 **Recursive Copy** (`cp -r`): Copies entire directory structures, preserving content.

Practical Script: Git Helper Wrapper

Small shell scripts like this can dramatically streamline daily development by automating repetitive tasks while still maintaining proper Git workflows. They combine multiple commands into a single, efficient utility.

```
git-quick.sh

1 #!/bin/bash
2 # git-quick.sh: Automate add, commit, and push.
3
4 # Usage: ./git-quick.sh "Your commit message"
5
6 msg="$1"
7
8 # Check if a commit message was provided
9 if [ -z "$msg" ]; then
10     echo "Error: Commit message is required."
11     echo "Usage: $0 \"<commit message>\""
12     exit 1
13 fi
14
15 echo "Starting Git Helper Wrapper..."

# Step 1: Stage all changes
echo "-> git add -A"
git add -A
if [ $? -ne 0 ]; then
    echo "Error during 'git add'. Exiting."
    exit 1
fi

# Step 2: Commit changes
echo "-> git commit -m \"$msg\""
git commit -m "$msg"
if [ $? -ne 0 ]; then
    echo "Error during 'git commit'. Exiting."
    exit 1
fi

# Step 3: Push changes
echo "-> git push"
git push
if [ $? -ne 0 ]; then
    echo "Error during 'git push'. Exiting."
    exit 1
fi

echo "Git Helper Wrapper completed successfully!"
```

BEFORE / AFTER COMPARISON

BEFORE (Manual Workflow)

```
user@host:~/project$ git add -A
(Staging all changes...)
[Success]

user@host:~/project$ git commit -m "Refactor API for better performance"
[master 882e9f0] Refactor API for better performance
 5 files changed, 120 insertions(+), 45 deletions(-)

user@host:~/project$ git push
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.45 KiB | 1.45 MiB/s, done.
Total 18 (delta 3), reused 0 (delta 0)
To https://github.com/user/project.git
c601a8e..882e9f0 master -> master
```

Multiple steps,
time-consuming



Sequential,
Manual Process

AFTER (Using Helper Wrapper)

```
user@host:~/project$ ./git-quick.sh 'Refactor API'
Starting Git Helper Wrapper...
-> git add -A
-> git commit -m "Refactor API"
[master a582c3d] Refactor API
 5 files changed, 120 insertions(+), 45 deletions(-)
-> git push
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.45 KiB | 1.45 MiB/s, done.
Total 18 (delta 3), reused 0 (delta 0)
To https://github.com/user/project.git
8dfe8f0..a582c3d master -> master
Git Helper Wrapper completed successfully!
```



**SIGNIFICANT
TIME SAVED!**



**Streamlined, Single
Command Execution**